

Compact Representations of Human Interception Movements

Simaan Libbiss and Paz Flashner

Workshop on Deep Learning, Tel Aviv University

Research supervision: Prof. Jason Friedman | Course advisor: Moni Shahar

Working draft. Red brackets identify questions for Jason.

1 Study objective and experimental data

We investigated whether a compact numerical fingerprint could capture individual differences in interception movements and reproduce a participant's movement distributions. The principal setting used three latent values, with eight providing a higher-capacity comparison. Neural encoder-decoder models were compared with spline-based dimensionality reduction and regression on reconstruction, feature distributions and individual information. These comparisons inform model selection for an interactive dashboard.

1.1 Experimental task

The recordings were provided by Jason Friedman's laboratory. Participants moved their finger from a starting region toward a fixed interception region. A circular target appeared on either side of the display and subsequently moved horizontally toward the interception region. Participants were instructed to reach that region while the moving target was inside it. Successful performance therefore required coordinating finger arrival with target arrival (Figure 1).

The target appeared at one of six starting locations: three on the left and three on the right. Each starting-position category was associated with a range of speeds, from which a continuous target speed was randomly selected for each trial. Starting position and speed were therefore related by the design, rather than varied independently.

Table 1. Nominal stimulus settings supplied by the laboratory; every category occurred on both sides. [Jason: Are the starting-position values in mm, and relative to which reference?]

Category	Starting-position setting	Target-speed range
1	120	255-300 mm/s
2	140	298-350 mm/s
3	160	340-400 mm/s

The experiment included central-fixation and free-eye-movement conditions. This study analyzed only the free-eye-movement condition (condition 2), as specified by the laboratory. Within this subset, the modeling variables were starting-position category, starting side and executed target speed.

1.2 Recordings and analysis sample

Finger position was recorded in three dimensions at 240 Hz using a Polhemus Liberty magnetic motion-capture system. Each CSV contained positions, sensor orientation, a frame counter, a reception timestamp and an event marker. Orientation was excluded. Frame counters supplied the sampling time base because reception times need not follow a regular sampling interval. Paired MATLAB files supplied the target trajectory at 60 Hz, event times and outcome labels.

The available condition-2 dataset comprised 4,763 trials from 28 participants. Four trials without an arrival event and 27 with arrival more than one second after the target window ended were excluded, leaving 4,732 trials. Early-arrival and fixation-warning labels were retained. [Jason: Confirm the late-arrival exclusion and retention of fixation-warning trials in condition 2.]

Twenty-six participants contributed 171-180 retained trials each; the other two contributed 60 and 28. These two participants had only 60 and 29 recordings available, respectively. Thus, the small counts mainly reflect incomplete available sessions rather than extensive exclusion. The median retained count was 179.5. [Jason: Are the missing recordings for subject02 and subject41 unavailable files or incomplete sessions, and should both participants remain in the analysis?]

1.3 Task and recorded-movement examples



Figure 1. Original free-eye-movement instruction screen supplied with the stimulus files (instructions2.jpg). The lower red square indicates the starting region; the upper blue square is the interception region. The instructions require finger arrival while the moving target occupies that region and permit free eye movements. This is the task display, not a photograph of the apparatus.

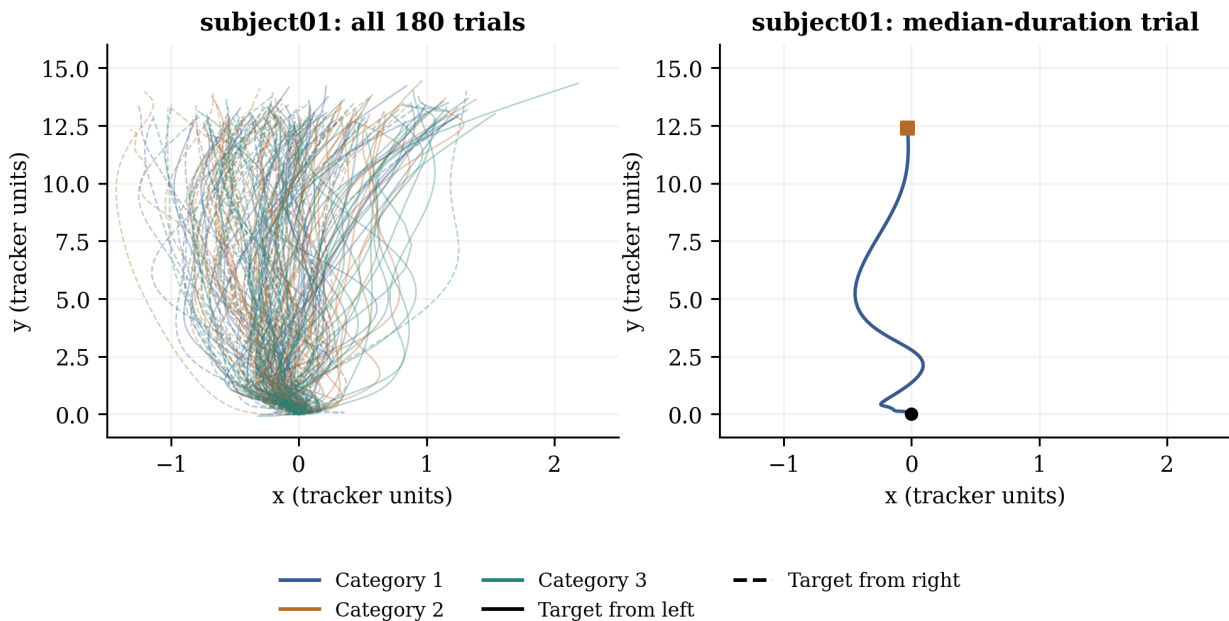


Figure 2. Recorded movements from subject01, selected as the first participant in identifier order. Left: all 180 retained trials, colored by target starting category; line style indicates target side. Right: one trial selected by median modeled duration, before inspecting model errors. Positions are filtered, aligned to target motion and translated to a common starting point. Panels share axis limits, with lateral variation enlarged relative to forward displacement. These are recorded data, not model outputs.

Repeated trials provide the observations from which a participant-specific distribution can be estimated. The spatial paths illustrate variation in the recordings, while the subsequent evaluation also examines initiation time, movement duration and derived kinematic features. Figure 3 shows how an individual recording enters the models.

2 From recordings to model inputs

Trial durations and the number of recorded samples varied. To compare and compress trajectories with a common model, we mapped every retained recording to 100 positions spanning the same relative interval within the trial. This temporal normalization was applied after establishing a regular sampling grid, filtering and selecting the modeled window.

2.1 Sampling grid, filtering and event alignment

Samples sharing a frame counter were averaged, and missing counters were filled by linear interpolation. The resulting grid had a 1/240 s interval. A fourth-order Butterworth low-pass filter with a 10 Hz cutoff was applied forward and backward to the three position coordinates. Filtering attenuated rapid fluctuations before resampling and speed-based onset detection.

Marker 5 identified target appearance. Target-motion onset was located from the first nonzero displacement in the executed target trajectory and mapped to the nearest tracker frame. The modeled window ran from that frame to the final recorded tracker sample. It therefore included waiting after target motion began. [Jason: Confirm the target-motion alignment and whether the earlier appearance-to-motion interval should also be modeled.]

The final tracker sample served as the arrival proxy; the event audit placed it, on average, 26.2 ms before the MATLAB arrival event (SD 3.6 ms). [Jason: Confirm the arrival-event meaning and whether this endpoint is appropriate.]

2.2 Temporal and spatial normalization

For a window containing L samples, sample i was assigned phase $u_i = i/(L - 1)$. A cubic interpolant S was fitted through the filtered coordinates and evaluated at 100 phases $u_j = j/99$, including both endpoints. Subtracting the first position then aligned the start of every trajectory:

$$\mathbf{P}_j = \mathbf{S}(j/99) - \mathbf{S}(0), \quad j = 0, \dots, 99.$$

This operation changes the sampling grid without fitting the later five-knot model. A phase of 0.5 denotes halfway through the selected time interval, not halfway along the traveled path. The resulting points preserve relative temporal progression, but the array alone does not specify duration in seconds. Spatial normalization translates the path; it does not rotate it or scale its amplitude.

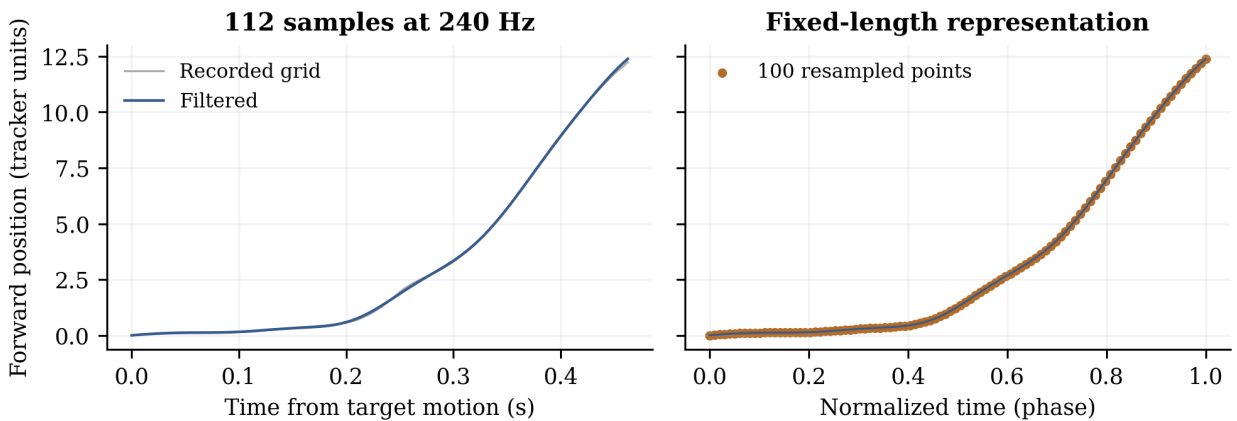


Figure 3. One subject01 trial before and after temporal normalization. The filtered forward-position trace uses physical elapsed time on the left. The right panel shows the same trace against phase, with all 100 resampled points. The same operation is applied to the other coordinates. Resampling does not create 100 independent observations.

The analysis used x and y from the normalized 3-D trajectory, producing a 100×2 matrix $\mathbf{P}$, flattened in time order into 200 neural input values. [Jason: Confirm that x - y is the task plane and confirm tracker position units (mm or cm).] Until confirmed, trajectory errors are reported in squared tracker units.

2.3 Duration targets and participant separation

Physical timing was retained separately from the phase-normalized coordinates. Finger onset was the first post-target-motion sample with 3-D speed above 5 tracker units/s for three consecutive frames. Initiation time was measured from target motion to this onset; movement time was measured from onset to the final sample. If no sustained crossing was found, the implementation used target motion as onset. [Jason: Confirm the onset threshold, persistence rule and fallback.]

The two durations were transformed as $\log(t + 0.001)$, with t in seconds, and standardized using training-trial means and standard deviations. They were supervised prediction targets and were not supplied as encoder inputs. Predictions were converted back to seconds by reversing standardization and the log transform, with negative values clipped to zero.

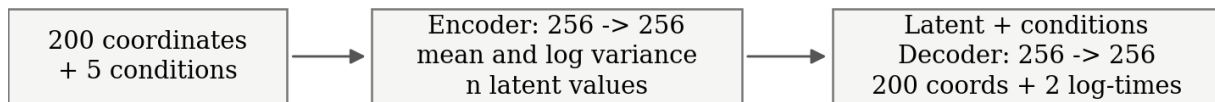
Four fixed participant folds contained 17 training, four validation and seven test participants each. Every participant appeared in a test fold once. Parameters and normalization were fitted on training participants; validation participants determined hyperparameters and checkpoints. Neural training used inverse-trial-count sampling to give each training participant equal expected sampling weight. Reported errors were averaged within each test participant before averaging participants equally.

Reconstruction used each complete held-out trajectory as an input and compared its decoded output with that trajectory. For participant fingerprints and generation, test trials were instead divided into disjoint, condition-stratified context and query sets. Only context codes contributed to the fingerprint; query movements supplied the evaluation distribution. These distinct input requirements were maintained throughout the comparisons.

3 Compact trajectory models

Spline + PCA and the neural encoders provided alternative mappings from the same observed trajectory to n latent values. The primary comparison used $n=3$, with $n=8$ testing a less restrictive representation. CVAE and spline runs also included $n=2$ and $n=4$; the matched neural ablations were evaluated at three and eight dimensions. The spline model supplied a smooth linear representation, while the neural models learned nonlinear encoder-decoder mappings.

Conditional variational autoencoder



Spline + PCA

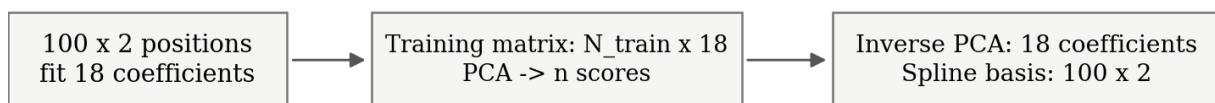


Figure 4. Alternative model pipelines. Each representation compresses a single trial to n values. A participant fingerprint is subsequently formed by averaging context-trial codes. The spline timing regression is separate from its shape reconstruction.

3.1 Spline fitting and PCA compression

The spline representation first replaced each 100-point coordinate sequence with a fitted smooth curve. We used cubic B-splines with five fixed interior knots at phases 1/6, 2/6, 3/6, 4/6 and 5/6. These define nine basis functions per coordinate. The knot locations were shared across trials; the fitted coefficients varied between trials.

Let B be the 100×9 matrix of basis values at the resampling phases. For each trial, the nine x coefficients and nine y coefficients were obtained by least-squares fitting:

$$\mathbf{c}_x = \arg \min_{\mathbf{c}} \|\mathbf{x} - B\mathbf{c}\|_2^2, \quad \mathbf{c}_y = \arg \min_{\mathbf{c}} \|\mathbf{y} - B\mathbf{c}\|_2^2.$$

Concatenating the coordinate coefficients produced an 18-value vector $\mathbf{c}$ per trial. Across training trials, these vectors formed an $N_{\text{train}} \times 18$ matrix. PCA was fitted to this matrix to learn a shared n -dimensional coefficient subspace. Raw and coefficient-standardized variants were compared using validation trajectory MSE, and the lower-error variant was retained. The PCA basis and scaling statistics were never fitted to test participants.

Writing $\mathbf{m}$ for the training coefficient mean, $\mathbf{d}$ for the coefficient scaling vector and V for the $18 \times n$ principal-component matrix, encoding and reconstruction can be expressed as:

$$\mathbf{z} = V^T[(\mathbf{c} - \mathbf{m}) \oslash \mathbf{d}], \quad \hat{\mathbf{c}} = \mathbf{m} + \mathbf{d} \odot (V\mathbf{z}).$$

Division and multiplication by $\mathbf{d}$ act coordinate-wise; $\mathbf{d}$ equals one for the unstandardized variant. PCA selects directions of variance in coefficient space. Its fitting objective is therefore distinct from minimizing trajectory MSE directly, although trajectory MSE is used to select the scaling variant.

The reconstructed coefficient vector was separated into x and y blocks. Multiplying each reconstructed coefficient block by B recovered its coordinate sequence. Test trajectories were fitted to the fixed spline basis to obtain coefficients and then projected through the training-fitted PCA basis. Thus, the final trial code contained n values, not all 18 spline coefficients.

3.2 Ridge regression for spline timing

Shape-only PCA did not contain explicit duration targets. We therefore fitted a separate Ridge regression to predict the two standardized log-durations from the concatenation of the n PCA scores and five task-condition values. Its input matrix had N_{train} rows and $n+5$ columns, standardized using training statistics. For input matrix U and target matrix Y , the fitted weights W and intercept b minimized:

$$\|Y - UW - \mathbf{1}b^T\|_F^2 + \alpha \|W\|_F^2.$$

The penalty reduced the size of regression coefficients; the intercept was unpenalized. Alpha was selected from 0.01, 0.1, 1, 10 and 100 by validation MSE in standardized log-time space. The selected regression was fitted on training participants and inverted to seconds at prediction time. This regression predicts timing; inverse PCA and the spline basis reconstruct the spatial trajectory.

3.3 Neural encoder-decoder models

The conditional variational autoencoder (CVAE) learned a nonlinear trajectory representation within the variational encoder-decoder framework [1,2]. The 200-coordinate input was standardized using a separate training mean and standard deviation for each coordinate. Five condition values were concatenated to it: three one-hot starting-category indicators, a binary starting-side value and continuous executed target speed. Speed was calculated from the target trajectory and divided by 1920 pixels to express it in screen widths/s. [Jason: Confirm the 1920-pixel screen-width assumption.]

Two fully connected layers of 256 ReLU units produced an n-dimensional posterior mean and log-variance vector. Training drew a latent sample through Gaussian reparameterization. The decoder concatenated z with the five conditions and applied two further 256-unit ReLU layers. Separate linear heads predicted 200 standardized coordinates and two standardized log-durations.

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

The per-trial objective combined coordinate reconstruction, duration prediction and regularization of the approximate posterior toward a standard-normal prior:

$$\mathcal{L} = \|\mathbf{v} - \hat{\mathbf{v}}\|_2^2 + 20\|\mathbf{y} - \hat{\mathbf{y}}\|_2^2 + \beta D_{\text{KL}}[q(\mathbf{z} | \mathbf{v}, \mathbf{c}) \| \mathcal{N}(\mathbf{0}, \mathbf{I})].$$

Here $\mathbf{v}$ denotes standardized trajectory coordinates, $\mathbf{y}$ the two standardized log-duration targets, and $\mathbf{c}$ the condition vector. Coordinate and timing errors are sums over their respective outputs; the complete loss is averaged over the batch. For the diagonal Gaussian posterior, the KL term sums contributions from all n coordinates. Beta increased from zero to one in 50 increments, reaching one at epoch 51.

$$D_{\text{KL}} = \frac{1}{2} \sum_{k=1}^n (\mu_k^2 + \sigma_k^2 - 1 - \log \sigma_k^2).$$

Training used Adam with initial learning rate 0.001, batches of 64 and gradient clipping at norm 5. Validation controlled learning-rate reduction and checkpoint selection using the full-target-KL objective. Training stopped after 25 epochs without improvement or at 150 epochs. Seeds 42, 43 and 44 repeated training. For deterministic reconstruction, the decoder received the posterior mean rather than a sampled code; its outputs were then returned to the original coordinate and timing scales.

Two ablations isolated changes to this architecture. Conditional AE (CAE) retained the condition inputs but replaced stochastic sampling with a deterministic code and removed the KL term. Unconditional VAE retained sampling and KL but replaced all condition inputs with zeros at both encoder and decoder. Its mathematical formulation consequently omitted the dependence on $\mathbf{c}$. The same trajectory and timing targets were used in these comparisons.

3.4 Condition-only control and dashboard scope

A separate condition-only Ridge model predicted the 200 trajectory coordinates from the five task values. It received neither the observed trajectory nor a participant fingerprint, providing a reference for prediction from task information alone. This differs from the spline timing regression, whose inputs include a trajectory code.

The dashboard supports CVAE and spline + PCA at $n=2, 3, 4$ and 8, and VAE and CAE at $n=3$ and 8. VAE at $n=8$ is the default for distribution exploration because it has the lowest cohort-average KS, energy and MMD-squared distances in the tested matrix (Section 5). This endpoint-specific numerical ranking does not imply superiority on every task. Conditions are inactive for VAE and affect only timing for spline + PCA. No $n=16$ result is included in the matched study.

4 Trajectory reconstruction results

The first evaluation asked how much of an observed trajectory each compact representation could recover. Both the spline and neural models received the complete held-out trajectory, compressed it to n values and decoded it to the original 100×2 shape. Reconstruction MSE was calculated as:

$$\text{MSE} = \frac{1}{200} \sum_{j=0}^{99} [(x_j - \hat{x}_j)^2 + (y_j - \hat{y}_j)^2].$$

Trial errors were averaged within participants, then across neural seeds and finally across the 28 participants with equal weight. This preserves each participant as the reporting unit despite unequal trial counts. Spline fits were deterministic within each fold and dimension. Errors are in squared tracker units.

Table 2. Participant-balanced reconstruction error. Lower is better. Condition-only Ridge has no latent dimension; its result is repeated for reference.

Model	n=3 MSE	n=8 MSE
Spline + PCA	0.097255	0.022772
VAE	0.148299	0.033487
Conditional AE	0.169661	0.025968
CVAE	0.176981	0.032481
Condition-only Ridge	1.233795	1.233795

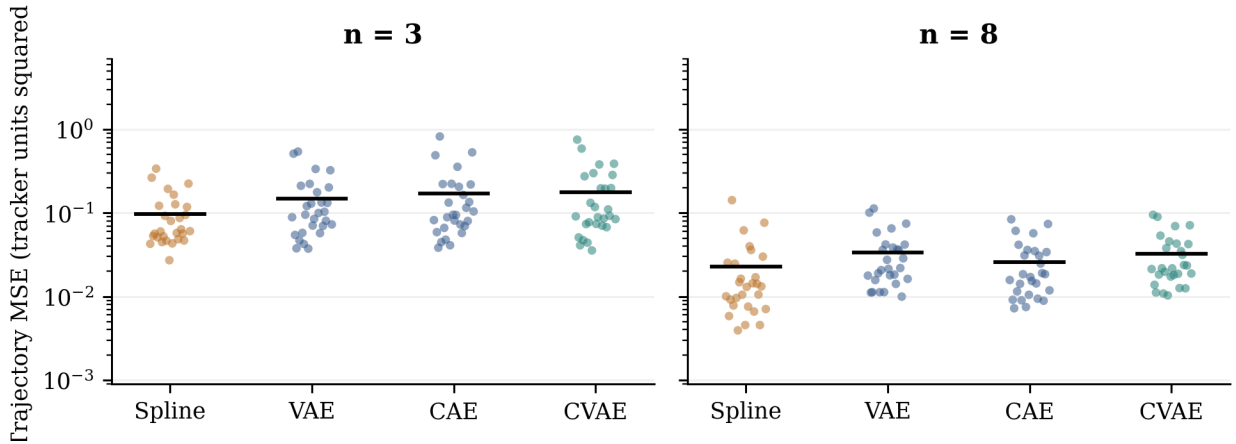


Figure 5. Participant errors after neural-seed averaging. Each point represents one participant; black bars show cohort means. The vertical axis is logarithmic.

Spline + PCA achieved the lowest mean reconstruction error at both capacities. Among neural models, VAE was best at $n=3$ (0.148299), whereas CAE was best at $n=8$ (0.025968). CVAE error decreased from 0.176981 to 0.032481, and spline error decreased from 0.097255 to 0.022772, when capacity increased from three to eight values.

Exploratory two-sided paired Wilcoxon tests used 28 seed-averaged participant values, Pratt zero handling and differences rounded to 12 decimals [3]. Benjamini-Hochberg adjustment covered 140 comparisons: ten model pairs, two dimensions and seven endpoints. At $n=3$, VAE had lower MSE than CVAE ($q=0.00163$) and CAE ($q=0.03695$). At $n=8$, CAE had lower MSE than both variational models ($q<0.000001$). All six spline-neural contrasts favored spline after this adjustment.

Holm adjustment across the same 140 tests retained the two $n=8$ neural contrasts but not the two $n=3$ contrasts. Because folds share development participants, these tests provide exploratory within-cohort evidence rather than independent population confirmation [4].

4.1 Recorded and reconstructed trajectories

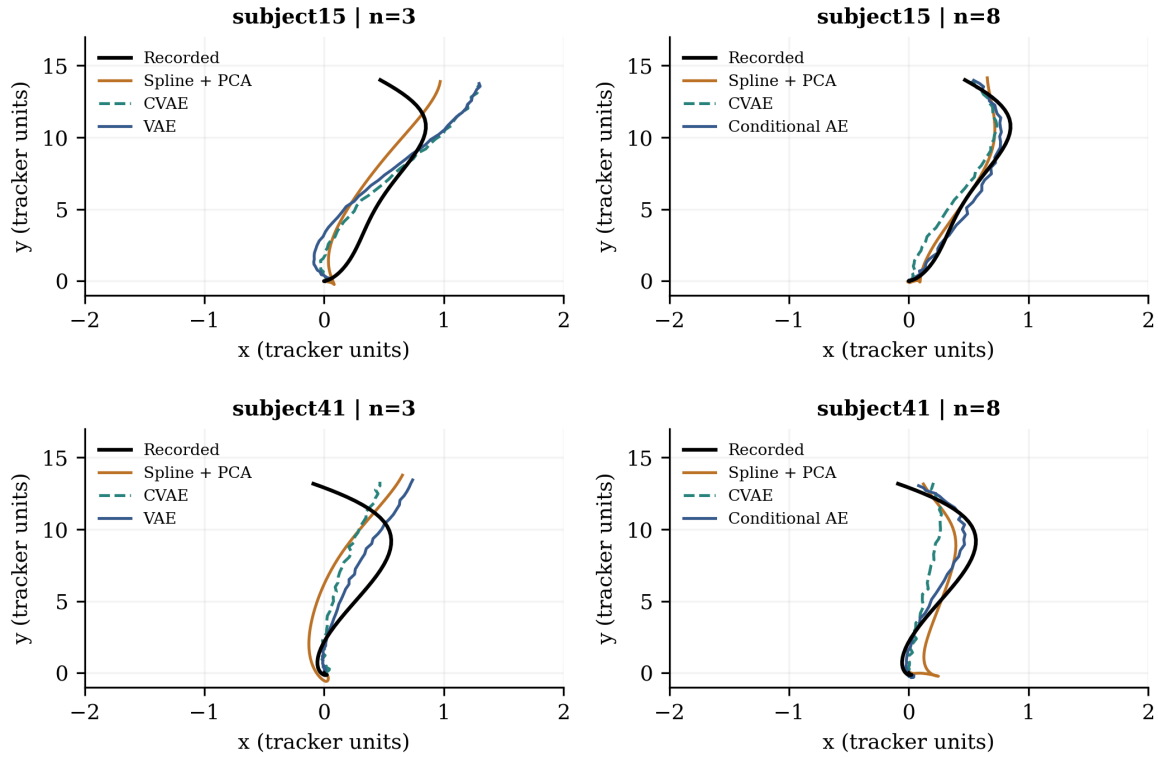


Figure 6. Recorded inputs and decoded outputs, using neural seed 42. Blue represents VAE at $n=3$ and CAE at $n=8$. All panels share axis limits; the lateral axis is expanded relative to the forward axis to reveal differences. These are reconstructions, not newly generated movements.

Examples were chosen by a fixed error-ranking rule. Participants were ranked by mean $n=3$ reconstruction MSE across the four compression families, averaging neural seeds first. The 14th and 26th participants represented the central and higher-error portions of the ranking. Within each participant, the median-ranked trial under the same family-averaged error was selected. The same input was displayed at both capacities.

The selected trials were **subject15_li_2_1_1_26** (fold 2); **subject41_li_2_1_1_25** (fold 1). Selection used held-out errors for illustration only. The aggregate comparison includes all 28 participants; a model with the lowest cohort mean need not be best on each displayed trial.

Reconstruction establishes how accurately the models compress observed paths. The fingerprint objective requires a further test: whether a code estimated from one set of a participant's movements can reproduce the distribution of other movements. The subsequent sections therefore evaluate duration prediction, generated feature distributions and participant-specific information. Side-by-side generation figures will be labeled separately from these input-output reconstructions.

5 Generating participant-specific movement distributions

Accurate reconstruction shows that a code retains information about an observed movement. The fingerprint objective asks whether a code obtained from a participant's previous trials can also describe their other movements. We therefore evaluated generation using disjoint context and query trials from participants excluded from model development. Because generated movements have no unique recorded counterpart, this evaluation compared distributions rather than assigning a pointwise reconstruction error to each generated trajectory.

5.1 Context enrollment and stochastic generation

Within each held-out participant, trials were divided approximately equally into context and query sets, stratified by starting category and side. A fixed partition was shared across models. Each context trajectory was encoded, and its latent code contributed to the participant fingerprint. For variational models the code was the posterior mean; CAE used its deterministic encoding and spline + PCA used its PCA scores. The fingerprint was the mean of these context codes:

$$\theta_s = \frac{1}{|C_s|} \sum_{i \in C_s} \mu_i, \quad \mathbf{z}^{(b)} \sim \mathcal{N}(\theta_s, \Sigma_{\text{train}}).$$

Here C denotes the participant's context set and the fingerprint has n values. Trial variability was supplied by a shared covariance estimated from training participants, allowing multiple latent samples to be drawn around that fingerprint. For the neural models, this covariance combined variation in trial codes around each training participant's mean with the mean encoder posterior variance:

$$\Sigma_{\text{train}} = \text{Cov}_{\text{train}}(\mu_i - \bar{\mu}_{s(i)}) + \text{diag}(\mathbb{E}_{\text{train}}[\sigma_i^2]) + 10^{-6}I.$$

The small diagonal term stabilizes sampling. For CAE, the posterior-variance contribution is approximately zero. Spline + PCA used the covariance of training scores after subtracting each training participant's mean, with the same diagonal stabilizer. Consequently, individual context trials determine the fingerprint center, while the covariance is shared. The n -value fingerprint does not independently estimate each participant's variability or covariance structure.

For each participant and fitted model, 120 latent samples were decoded into trajectories and durations. CVAE and CAE received task-condition vectors drawn from the participant's query-condition mixture; VAE ignored those vectors. Query task metadata supplied the mixture being evaluated, while query trajectories were withheld from fingerprint construction and generation. For spline + PCA, sampled scores were decoded through inverse PCA and the spline basis, and the timing Ridge received those scores and conditions. Its spatial decoder did not depend on conditions.

The generated outputs were therefore complete phase-normalized trajectories accompanied by predicted movement and initiation times. The evaluation reflects generation under the observed condition mixture. It does not establish extrapolation to new task conditions or adaptation across recording sessions.

5.2 Movement features

We extracted 11 timing, speed and geometry features (Table 3). Recorded features used the observed movement-onset-to-end segment. Generated segments used predicted initiation as a fraction of total duration, capped at 0.98, followed by resampling to 100 points. Movement duration was floored at 1 ms; initiation time at zero.

5.2 Movement features (continued)

Table 3. The 11 generated-feature endpoints. Speed-peak count is a heuristic descriptor and is distinct from the minimum-jerk component count evaluated later.

Feature	Definition and unit
Initiation time	Target-motion onset to finger onset; seconds.
Movement time	Finger onset to the final sample; seconds.
Peak speed	Maximum norm of the position gradient; tracker units/s.
Time to peak speed	Peak-speed sample index divided by the number of movement intervals; fraction.
Path length	Sum of consecutive position-step lengths; tracker units.
Straight-line distance	Distance from movement start to movement end; tracker units.
Curvature index	Path length divided by straight-line distance; dimensionless.
Maximum lateral deviation	Largest perpendicular distance to the start-to-end line; tracker units.
Speed-peak count	Peaks with prominence at least 10% of the maximum and separation of about 50 ms; minimum reported count is one.
Endpoint x; endpoint y	Two features: final movement coordinates relative to movement onset; tracker units.

Both recorded and generated movement segments passed through the same feature function. Positions were low-pass filtered with a cutoff of min(10 Hz, 0.45 times the effective sampling rate), and finite differences produced velocity. The effective rate was 99 divided by movement duration in seconds. The feature code was shared, although recorded onset and timing were measured and generated onset and timing were predicted. The recorded segments also inherited acquisition preprocessing; this comparison does not remove that upstream difference.

5.3 Distribution comparisons

For each participant and feature, the two-sample Kolmogorov-Smirnov statistic [3] measured the largest difference between the recorded-query and generated empirical cumulative distributions. The mean of the 11 statistics summarized marginal feature fidelity:

$$D_{sf} = \sup_x |F_{sf}(x) - G_{sf}(x)|, \quad \overline{D}_s = \frac{1}{11} \sum_{f=1}^{11} D_{sf}.$$

Lower values indicate closer distributions. We used KS as a descriptive distance, including for discrete peak counts; continuous-distribution KS p-values were not used to infer fidelity of that discrete feature. Matching marginal distributions alone does not ensure that relationships among features are reproduced.

Energy discrepancy and squared maximum mean discrepancy (MMD) [5] therefore compared complete 11-feature vectors. Features were centered and scaled using training data, with a scale floor of one for peak counts. The energy statistic was twice the average cross-sample Euclidean distance minus the two average within-sample distances:

$$E = 2 \overline{\|X - Y\|} - \overline{\|X - X'\|} - \overline{\|Y - Y'\|}.$$

These averages included all ordered pairs and within-sample diagonal zeros; the reported statistic was not square-rooted. MMD used a Gaussian kernel with bandwidth fixed from training data and excluded diagonal within-sample pairs:

$$\widehat{\text{MMD}}_u^2 = \frac{\sum_{i \neq j} k(X_i, X_j)}{m(m-1)} + \frac{\sum_{i \neq j} k(Y_i, Y_j)}{r(r-1)} - \frac{2 \sum_{i,j} k(X_i, Y_j)}{mr}.$$

Here m and r are recorded and generated sample counts. The kernel was exp(-gamma times squared Euclidean distance), with gamma set to the inverse median positive squared distance among a fixed training subset of up to 512 trials. This unbiased MMD-squared estimator can be slightly negative in finite samples. All distance references were shared across models within a fold.

5.4 Generated-feature results

Distances were computed separately for each test participant, averaged across available neural training seeds and then across the 28 participants with equal weight. Table 4 presents the four compact-representation families at matched capacities. Spline + PCA contributes a single deterministic fit and fixed sampling realization per fold and dimension; neural results summarize seeds 42, 43 and 44.

Table 4. Cohort-average generated-feature discrepancies. Lower is better. All three metrics compare distributions; none is a trajectory reconstruction MSE.

Model	n	Mean KS	Energy	MMD squared
Spline + PCA	3	0.325126	0.977433	0.181467
CVAE	3	0.277795	0.745431	0.130935
CAE	3	0.287578	0.801489	0.141410
VAE	3	0.284587	0.747566	0.130298
Spline + PCA	8	0.254794	0.734715	0.115291
CVAE	8	0.219404	0.430451	0.067493
CAE	8	0.258686	0.620224	0.111135
VAE	8	0.215121	0.411014	0.063881

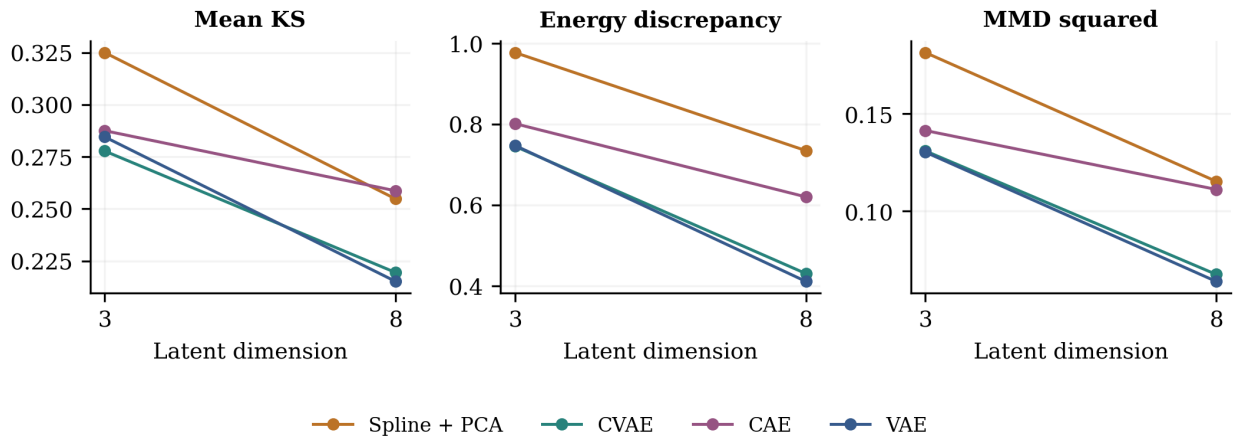


Figure 7. Generated-feature discrepancies at three and eight dimensions. Lines connect tested settings only and do not estimate performance at untested capacities.

The reconstruction ranking did not carry over unchanged to generation. At $n=3$, CVAE had the lowest mean KS (0.277795) and energy discrepancy (0.745431) among these four families, while VAE had the lowest MMD-squared value (0.130298). At $n=8$, VAE had the lowest mean on all three distances: KS 0.215121, energy 0.411014 and MMD squared 0.063881. Spline + PCA retained its reconstruction advantage but had larger generation discrepancies than VAE and CVAE at $n=8$.

Numerical advantages were not uniformly significant. Within the same exploratory 140-comparison family used for reconstruction, the $n=8$ VAE-CVAE contrast survived Benjamini-Hochberg adjustment for energy ($q=0.02575$), but not for mean KS ($q=0.13686$) or MMD squared ($q=0.05182$). None of these three contrasts survived Holm adjustment across 140 tests. The results support an endpoint-specific numerical preference rather than universal dominance.

VAE $n=8$ was consequently selected as the dashboard's default for marginal-distribution exploration. All other supported references remain selectable. The choice uses this evaluated cohort and has no separate post-selection validation. Three-dimensional models remain available for the original compact-fingerprint question; $n=16$ was not evaluated in the matched study. A higher-capacity model might improve a particular endpoint, but the present data do not establish that outcome.

5.5 Recorded and generated movement examples

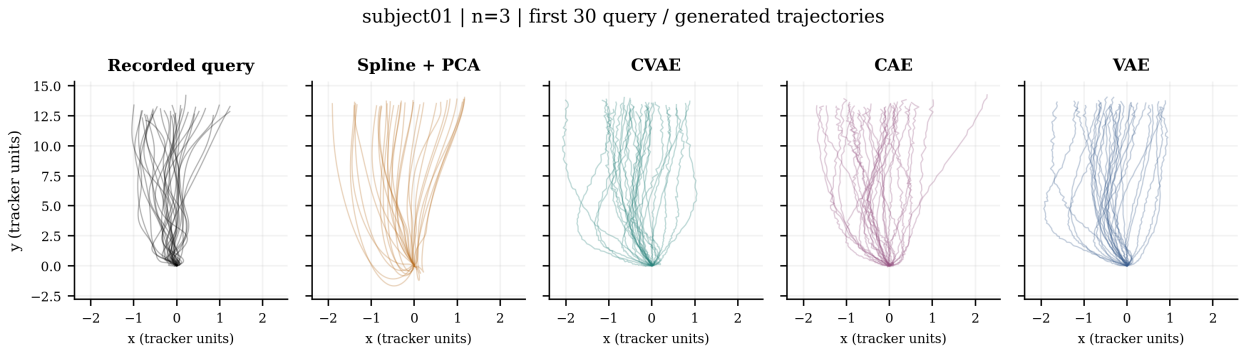


Figure 8. Recorded query trajectories and generated trajectories at $n=3$ for subject01. Each panel displays the first 30 available samples in its fixed ordering. Generated panels use context fingerprints, training-derived covariance and the seed-42 models. The recorded paths were not reconstruction inputs.

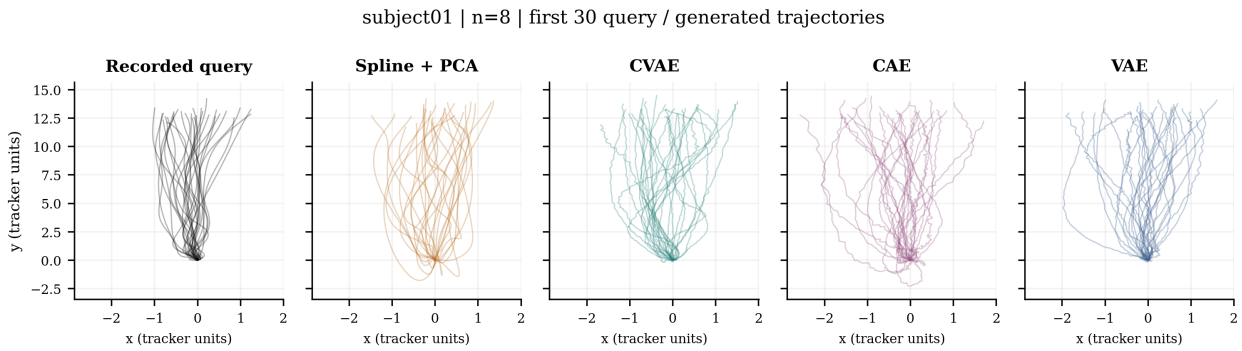


Figure 9. The corresponding $n=8$ samples for the same participant. Axis limits are shared across models within each figure and include every displayed point; Figures 8 and 9 can have different limits. Axes are not drawn at equal physical scale. No generated sample was selected for resembling a recorded trial.

The participant was selected as the first identifier in the analyzed cohort, independently of reconstruction or distribution scores. This participant was held out in fold 2. Each model generated 120 trajectories for evaluation; 30 are displayed to limit overplotting. The panels illustrate the range of decoded paths, while Table 4 summarizes the entire cohort and all evaluated seeds. Visual resemblance of individual examples is not evidence that two stochastic samples should coincide point by point.

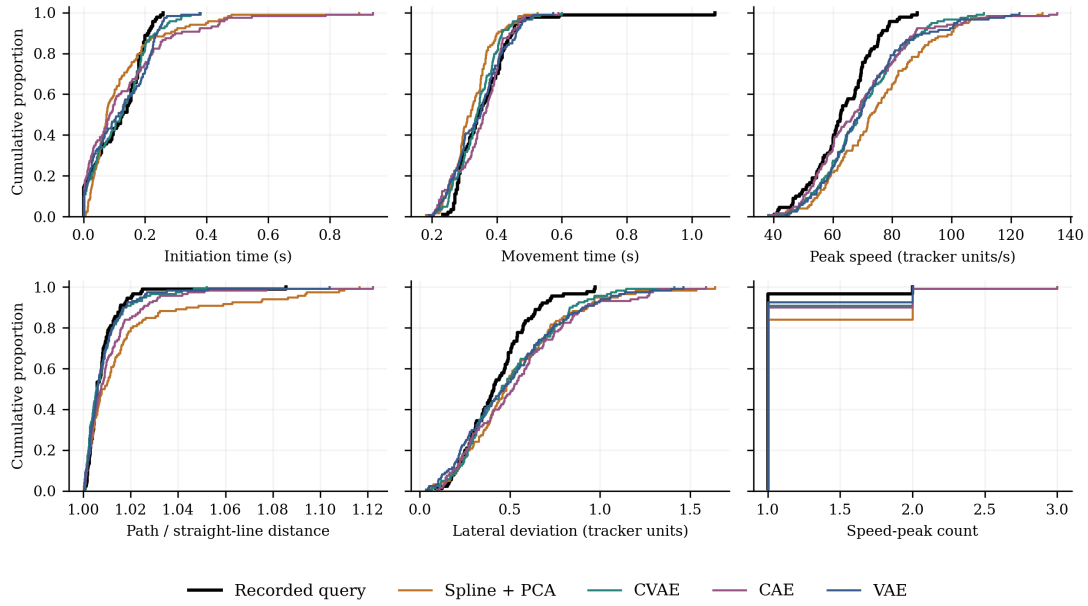


Figure 10. Six feature distributions for subject01 at $n=8$: recorded query trials and all 120 generated samples per model, using seed 42. These fixed display features cover timing, speed, geometry and peak count. Vertical separation between two cumulative curves illustrates a distribution discrepancy; Table 4 averages all 11 features and all participants.

These comparisons assess generation from personal context, but they do not yet isolate the contribution of the personal fingerprint. The next analysis holds the decoder and sampling procedure fixed while replacing the participant's context fingerprint with population and wrong-participant controls.

References

- [1] Kingma DP, Welling M. Auto-Encoding Variational Bayes. ICLR, 2014. arXiv:1312.6114.
- [2] Sohn K, Lee H, Yan X. Learning Structured Output Representation using Deep Conditional Generative Models. NeurIPS 28, 2015.
- [3] SciPy developers. `scipy.stats.wilcoxon` and `scipy.stats.ks_2samp` documentation. Evaluations executed with SciPy 1.16.3.
- [4] Bengio Y, Grandvalet Y. No unbiased estimator of the variance of K-fold cross-validation. JMLR. 2004;5:1089-1105.
- [5] Gretton A, Borgwardt KM, Rasch MJ, Scholkopf B, Smola A. A Kernel Two-Sample Test. JMLR. 2012;13:723-773. <https://www.jmlr.org/papers/v13/gretton12a.html>